

OLYMPUS · SECURITY REVIEW

Olympus Convertible Deposits.

A security review of Convertible Deposits.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Olympus	Convertible Deposits	Guardian	July 28 – August 18, 2025	September 16, 2025

§ 00

Contents.

115 pages — overview, confidence ranking, scope, 92 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 65 findings	P. 13
	Remediation Review · 27 findings	P. 84
§ 05	Disclaimer	P. 114
§ 06	About Guardian	P. 115

§ 01

Overview.

Guardian reviewed Convertible Deposits across 2 review rounds. The final report includes 1 Critical, 13 High, 13 Medium, 39 Low, 26 Informational findings.

AUDIT FIRM

Guardian

CLIENT

Olympus

PROJECT

Convertible Deposits

PREPARED BY

Guardian

AUDIT WINDOW

July 28 – August 18, 2025

FINAL REPORT

September 16, 2025

REPOSITORY

0lympusDA0/olympus-v3

METHODOLOGY

Smart contract · Manual · Main Review · Remediation Review

MAIN REVIEW COMMIT

08e562e6d4e4

REMEDIATION REVIEW COMMIT

c26798cf216e

1.1 Confidence Ranking

3 / 5

**Moderate
Confidence**

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer's assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
● Critical	1	0	0	0	1
● High	13	0	4	0	9
● Medium	13	0	6	0	7
● Low	39	0	15	0	24
● Informational	26	0	9	0	17
All severities	92	0	34	0	58

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence ASSIGNED	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Convertible Deposits in `0lympusDA0/o1ympus-v3`. The scope includes 3,198 source lines across 21 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
src/bases/BasePeriodicTaskManager.sol	94	192
src/bases/BaseAssetManager.sol	118	245
src/policies/ReserveWrapper.sol	70	140
src/policies/Heart.sol	116	218
src/policies/EmissionManager.sol	326	564
src/libraries/Uint256.sol	22	27
src/libraries/Timestamp.sol	33	43
src/libraries/ERC6909Wrappable.sol	125	221
src/libraries/DecimalString.sol	51	87
src/libraries/CloneableReceiptToken.sol	36	90
src/libraries/AddressStorageArray.sol	23	47
src/external/clones/CloneERC20.sol	64	125
src/modules/DEPOS/PositionTokenRenderer.sol	146	184
src/modules/DEPOS/OlympusDepositPositionManager.sol	207	432
src/modules/DEPOS/DEPOS.v1.sol	11	37
src/policies/deposits/YieldDepositFacility.sol	257	426
src/policies/deposits/DepositRedemptionVault.sol	434	764
src/policies/deposits/DepositManager.sol	295	548
src/policies/deposits/ConvertibleDepositFacility.sol	212	379
src/policies/deposits/ConvertibleDepositAuctioneer.sol	362	749
src/policies/deposits/BaseDepositFacility.sol	196	337

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	Incorrect Scaling Overmints OHM To User	Logical Error	CRITICAL	RESOLVED	p. 13
H-01	Reclaims Can Force Insolvent Deposit Managers	Logical Error	HIGH	RESOLVED	p. 14
H-02	Yield Claim DoS'ed By Share Inconsistencies	Logical Error	HIGH	ACKNOWLEDGED	p. 16
H-03	Split Positions Cannot Claim Yield	Logical Error	HIGH	RESOLVED	p. 17
H-04	Facilities Can Steal Yield From Each Other	Logical Error	HIGH	RESOLVED	p. 18
H-05	Auctioneer Incompatible	Logical Error	HIGH	RESOLVED	p. 19
H-06	Heart Beat DoS'ed On Zero Emissions	DoS	HIGH	RESOLVED	p. 20
H-07	User Can Game Rate Received From Vault	Gaming	HIGH	RESOLVED	p. 21
H-08	Insolvency Due To Fixed-Amount Withdrawals	Logical Error	HIGH	ACKNOWLEDGED	p. 22
M-01	Griefing Receipt Token Conversion	Access Control	MEDIUM	RESOLVED	p. 23
M-02	Withdraw Rounding Prevents Yield Claims	Logical Error	MEDIUM	RESOLVED	p. 24
M-03	Emissions Not Adjusted	Logical Error	MEDIUM	ACKNOWLEDGED	p. 25
M-04	Keepers Not Incentivized To Quickly Beat	Warning	MEDIUM	ACKNOWLEDGED	p. 26

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-05	Yield Lost For Expired Positions	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 27
M-06	Converted Reserves Do Not Increase Backing	Logical Error	MEDIUM	ACKNOWLEDGED	p. 28
M-07	First Depositor Attack Affects Redemptions	Logical Error	MEDIUM	ACKNOWLEDGED	p. 29
M-08	Capacity Can Grow Incorrectly	Unexpected Behavior	MEDIUM	RESOLVED	p. 31
M-09	Capacity Calculation Incorrect	Logical Error	MEDIUM	RESOLVED	p. 33
M-10	Target Update Without LastUpdate Adjustment	Logical Error	MEDIUM	RESOLVED	p. 34
M-11	Inconsistent Global VS Per-Period Auction Parameters	Logical Error	MEDIUM	RESOLVED	p. 35
L-01	Risk Of Underflow With Multiple Operators	Warning	LOW	RESOLVED	p. 37
L-02	Insolvency When Large Yield	Logical Error	LOW	RESOLVED	p. 39
L-03	Protocol Risk When Interest Exceeds Buffer	Warning	LOW	ACKNOWLEDGED	p. 40
L-04	User Can Delay Reclaims	Censoring	LOW	ACKNOWLEDGED	p. 41
L-05	Default VS Reclaim Gaming	Warning	LOW	RESOLVED	p. 42
L-06	Delayed Heartbeat Affects Auction Tracking	Warning	LOW	RESOLVED	p. 43
L-07	Price Decays Faster Than Expected	Unexpected Behavior	LOW	RESOLVED	p. 44
L-08	Deposit Period Enabled Before Policy	Warning	LOW	RESOLVED	p. 45
L-09	Re-enabling Loses Tick Data	Warning	LOW	RESOLVED	p. 46
L-10	Setting Tracking Period Wipes Existing Results	Warning	LOW	RESOLVED	p. 47

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-11	Failed Bond Market Creation Derails Auction	Warning	LOW	RESOLVED	p. 48
L-12	Disabling Policy Loses Auction Results	Warning	LOW	ACKNOWLEDGED	p. 49
L-13	Allowance Decrease Frontrunning	Warning	LOW	ACKNOWLEDGED	p. 50
L-14	Task Execution Succeeds	Warning	LOW	RESOLVED	p. 51
L-15	Max Yield Cannot Be Claimed	Warning	LOW	RESOLVED	p. 52
L-16	Cross-Contract Reentrancy Risk	Warning	LOW	RESOLVED	p. 53
L-17	Parameters Should Be Configurable	Warning	LOW	RESOLVED	p. 54
L-18	Lack Of Validation On Withdrawal	Warning	LOW	RESOLVED	p. 55
L-19	Missing Validation For previewReclaim	Validation	LOW	RESOLVED	p. 56
L-20	Initial Auction Parameters Not Validated	Validation	LOW	RESOLVED	p. 57
L-21	Risk-Free Bids	Warning	LOW	ACKNOWLEDGED	p. 58
L-22	Asymmetric Yield Across Multiple Claims	Gaming	LOW	RESOLVED	p. 59
L-23	ERC-4626 Deposit VS PreviewRedeem Invariant	Superfluous Code	LOW	ACKNOWLEDGED	p. 60
L-24	Vault Validation Warnings	Validation	LOW	ACKNOWLEDGED	p. 61
L-25	Asymmetric Remaining Deposit	Warning	LOW	RESOLVED	p. 62
L-26	Lack Of Slippage Protection On Bids	MEV	LOW	RESOLVED	p. 63
L-27	Split Griefing Increases Gas Cost	Gas Griefing	LOW	RESOLVED	p. 64
L-28	Potential Superior Loans	Configuration	LOW	RESOLVED	p. 65

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-29	Handle Loan Functions	Validation	LOW	RESOLVED	p. 66
L-30	Borrowing More Is Incentivized For Defaults	Warning	LOW	ACKNOWLEDGED	p. 68
I-01	YDF conversionPrice Should Use Constant	Best Practices	INFORMATIONAL	RESOLVED	p. 69
I-02	Empty Position Id List Validation Missing	Best Practices	INFORMATIONAL	RESOLVED	p. 70
I-03	depositAmount Calculation Rounds	Rounding	INFORMATIONAL	RESOLVED	p. 71
I-04	Unwrapped Receipt Necessary For Redemption	Documentation	INFORMATIONAL	RESOLVED	p. 72
I-05	Actual Amount Ignored During Bids	Informational	INFORMATIONAL	RESOLVED	p. 73
I-06	Redundant Boolean Check	Best Practices	INFORMATIONAL	RESOLVED	p. 74
I-07	Using Magic Numbers	Best Practices	INFORMATIONAL	RESOLVED	p. 75
I-08	Superfluous Permission Request	Superfluous Code	INFORMATIONAL	RESOLVED	p. 76
I-09	Borrows Pay Full Interest Regardless Of Payback	Documentation	INFORMATIONAL	RESOLVED	p. 77
I-10	Newly Minted OHM Not Counted In Supply	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 78
I-11	Convertible Deposit Facility (CDF)	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 79
I-12	Unnecessary Day State Storage Read	Gas Optimization	INFORMATIONAL	RESOLVED	p. 80
I-13	Redundant Setting Of Asset And Period	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 81
I-14	Unnecessary While Loop	Gas Optimization	INFORMATIONAL	ACKNOWLEDGED	p. 82
I-15	Lack Of SafeTransfer	Best Practices	INFORMATIONAL	RESOLVED	p. 83

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
REMEDIATION REVIEW					
H-01	Reclaim DoS When Actual Less Than Discounted	Rounding	HIGH	RESOLVED	p. 84
H-02	Borrowing Insolvency Prevents Borrows	Rounding	HIGH	RESOLVED	p. 85
H-03	Reclaims Can Force Insolvent Deposit Managers	Logical Error	HIGH	ACKNOWLEDGED	p. 86
H-04	Gas Intensive YDF Prevents Yield Claims	DoS	HIGH	ACKNOWLEDGED	p. 88
H-05	Auctions DoS'd By Increasing Price	DoS	HIGH	RESOLVED	p. 90
M-01	Claim Yield Revert With High Yield Fee	Logical Error	MEDIUM	ACKNOWLEDGED	p. 92
M-02	Split To Bypass Minimum Deposit	Logical Error	MEDIUM	RESOLVED	p. 93
L-01	Period Enabled in Auctioneer Is Disabled In DM	Validation	LOW	ACKNOWLEDGED	p. 94
L-02	Lack Of Access Control On Create Token	Access Control	LOW	ACKNOWLEDGED	p. 95
L-03	Minimum Deposit and Caps Not Granular	Best Practices	LOW	ACKNOWLEDGED	p. 96
L-04	Rates Can Change Across Block	Warning	LOW	ACKNOWLEDGED	p. 97
L-05	Zero Actual Amount Not Verified	Warning	LOW	RESOLVED	p. 98
L-06	Min Price Straddle	Warning	LOW	ACKNOWLEDGED	p. 99
L-07	Asymmetry Between Preview And Borrow Amounts	Rounding	LOW	ACKNOWLEDGED	p. 100
L-08	Queue Not Cleared When Policy Disabled	Warning	LOW	ACKNOWLEDGED	p. 101
L-09	No Single Source Of Truth For Deposit Periods	Logical Error	LOW	RESOLVED	p. 102

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-01	DepositManager_Insolvent Revert Needs More Data	Best Practices	INFORMATIONAL	RESOLVED	p. 103
I-02	Deposit Periods Can Be Alternated	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 104
I-03	Receipt Token Symbol Not Prefixed With "r"	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 105
I-04	Pending Queue Internal	Best Practices	INFORMATIONAL	RESOLVED	p. 106
I-05	Bond Markets Created When Auctioneer Disabled	Warning	INFORMATIONAL	RESOLVED	p. 107
I-06	Superfluous Enumerable Set Validation	Gas Optimization	INFORMATIONAL	RESOLVED	p. 108
I-07	Shadowed State Variable	Best Practices	INFORMATIONAL	RESOLVED	p. 109
I-08	Periods Should Be Enabled Before Policy	Configuration	INFORMATIONAL	RESOLVED	p. 110
I-09	Backing/premium jumps when Treasury temporarily holds raw reserves	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 111
I-10	Missing min-shares/slippage check on periodic vault deposits	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 112
I-11	ERC-6909 supply is not authoritative for receipt tokens	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 113

CRITICAL C-01

RESOLVED

Incorrect Scaling Overmints OHM To User

CATEGORY	Logical Error
LOCATION	https://github.com/OlympusDAO/olympus-v3/blob/08e562e6d4e4f3ef7bdf5553de99574403cfdbc9/src/policies/deposits/ConvertibleDepositFacility.sol#L221
REVIEW	Main Review

DESCRIPTION

In the conversion logic, `convertedTokenOut = FullMath.mulDiv(amount_, 10 ** IERC20(currentAsset).decimals(), position.conversionPrice)` multiplies `amount_` (reserve token amount) by the reserve token's decimals (e.g., 18 for USDS). However, `position.conversionPrice` is calculated as `depositIn.mulDivUp(_ohmScale, ohmOut)` where `_ohmScale = 1e9` for OHM's 9 decimals.

This introduces an extra factor of `10 ** reserveDecimals / 10 ** ohmDecimals` (e.g., 10^9 for reserve=18, OHM=9), resulting in `convertedTokenOut` being massively inflated.

For reserve tokens with 18 decimals, users receive 1 billion times more OHM than intended which causes catastrophic OHM supply dilution and violates the principle that each OHM is backed appropriately by treasury assets.

RECOMMENDATION

Multiply by `_ohmScale` instead of `10 ** IERC20(currentAsset).decimals()` in `_previewConvert`.

PROOF OF CONCEPT <https://github.com/GuardianOrg/olympus-v3-olympus-fuzz/pull/9>

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/98>

HIGH **H-01**

RESOLVED

Reclaims Can Force Insolvent Deposit Managers

CATEGORY Logical Error
LOCATION YieldDepositFacility.sol: 325
REVIEW Main Review

DESCRIPTION

A user can call function `reclaimFor` to reclaim their entire deposit, but this does not update the user's DEPOS position, allowing users to claim yield on non-existent assets after reclaiming.

This vector allows for the following scenario:

- (1) Alice deposits $1e18$ reserve tokens through `createPosition`
- (2) Alice then reclaims her entire deposit, receiving a discounted amount. The DepositManager now holds 0 vault shares, but Alice's yield-earning position remains.
- (3) Bob then proceeds to deposit into the vault through `createPosition`.
- (4) Yield accrues and a snapshot is taken.
- (5) Alice claims yield (`claimYield`) with her phantom position, receiving some reserve tokens.
- (6) Bob attempts to claim yield, but because Alice already withdrew some assets, Bob's claim yield attempt reverts with `DepositManager_Insolvent`. Bob cannot claim the yield he has earned with the assets he deposited into the vault and that are generating yield in the vault.

Ultimately, Alice is able to prevent other users from claiming their yield by forcing insolvency through a reclaim, and also claim yield without having actively deposited assets in the vault.

Also note that the smaller the discount is configured (reclaim rate can be as high as 100%), the faster Alice will earn enough yield to overcome any immediate loss.

RECOMMENDATION

Adjust the remaining deposit of existing positions to reflect the withdrawal. Furthermore, ensure the reclaim rate is large enough to discourage malicious behavior.

H-01 Reclaims Can Force Insolvent Deposit ManagersCONTINUED

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/519e5506c2cc18add878ee00bc2dabcb>

— RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/86>

HIGH **H-02**

ACKNOWLEDGED

Yield Claim DoS'ed By Share Inconsistencies

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/master/src/policies/deposits/YieldDepositFacility.sol#L321
REVIEW	Main Review

— DESCRIPTION

Users are able to create a position in `YieldDepositFacility` in order to claim the yield from an ERC4626 vault strategy, without the ability to convert to OHM.

Furthermore, users holding receipt tokens can start a redemption and borrow against it, withdrawing funds from the ERC4626 vault. The main issue arises when users borrow using the `YieldDepositFacility`, as it reduces the active shares for the operator, while the facility's user's `lastShares` will still be using the full deposited value (sum of `lastShares` of users is greater than the real active shares)

This will impact the claimable yield calculation as users will try to claim more yield than what the max claimable yield allowed in the `DepositManager` for the `YieldDepositFacility`, DoS'ing users when calling `claimYield`.

Although this issue describes only `YieldDepositFacility` users, the same issue will apply if `ConvertibleDepositFacility` users redeem and borrow using the `YieldDepositFacility`.

— RECOMMENDATION

The solution requires a major refactor, as there is a discrepancy between the individual user shares calculation in `YieldDepositFacility` vs the actual shares of the facility in the `DepositManager`. When funds are borrowed out from the vault, these will not accrue yield, which should be reflected in the `_previewClaimYield` calculation.

PROOF OF CONCEPT <https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/guardian/src/test/guardian/pocs/YieldReducedFromBorrows.sol>

— RESOLUTION

Status: Acknowledged.

HIGH **H-03**

RESOLVED

Split Positions Cannot Claim Yield

CATEGORY	Logical Error
LOCATION	OlympusDepositPositionManager.sol: 254
REVIEW	Main Review

DESCRIPTION

When a position is created in YieldDepositFacility, `positionLastYieldConversionRate` is set to ensure yield is claimed only from the deposit time onward.

However, when splitting a position in `OlympusDepositPositionManager::split`, the new position's `positionLastYieldConversionRate` is not set, leaving it at 0.

This causes a division-by-zero revert in `claimYield` when calculating `lastShares = mulDiv(remainingDeposit, decimals, lastSnapshotRate = 0)`, blocking yield claims for the new position.

RECOMMENDATION

Stamp `positionLastYieldConversionRate` when splitting for YDF positions.

PROOF OF CONCEPT <https://github.com/GuardianOrg/olympus-v3-olympus-team1/pull/1>

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/90>

HIGH **H-04**

RESOLVED

Facilities Can Steal Yield From Each Other

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/master/src/policies/deposits/DepositRedemptionVault.sol#L220
REVIEW	Main Review

— DESCRIPTION

Receipt tokens are fungible (receipt tokens created by one operator can be reclaimed through another). Therefore, `ConvertibleDepositFacility` depositors can reclaim their tokens using the `YieldDepositFacility`.

However, reclaiming receipt tokens reduce the amount of yield earned by the facility as it redeems shares from the operator. In extreme cases, the `YieldDepositFacility` can be left with zero operator shares, preventing users to claim yield.

On the other side, claiming yield from the `ConvertibleDepositFacility` will be possible, but it is transferred to the treasury.

— RECOMMENDATION

Prevent users from the `ConvertibleDepositFacility` to be able to reclaim using a different facility (`YieldDepositFacility`).

PROOF OF CONCEPT <https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/guardian/src/test/guardian/pocs/CrossFacilityReclaimAffectsYield.sol>

— RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/86>

HIGH **H-05**

RESOLVED

Auctioneer Incompatible

CATEGORY	Logical Error
LOCATION	ConvertibleDepositAuctioneer.sol
REVIEW	Main Review

DESCRIPTION

The Auctioneer calculates the OHM output as `convertibleAmount = deposit_.mulDiv(_ohmScale, price_);`

This assumes price has deposit token decimal precision, but the price provided through `PRICE.getCurrentPrice()` in the EmissionManager is 18 decimals. Non-18 decimal deposit tokens are incompatible with the Auctioneer since the resulting `convertibleAmount` is no longer a 9 decimal OHM output amount. This will lead to the output amount to be larger than expected and absorb more capacity than expected, or truncate down to 0 when dealing with smaller decimal tokens such as USDC and revert.

RECOMMENDATION

Scale the price to the deposit token's decimals.

PROOF OF CONCEPT <https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/55102a5a9818806a74c0701f8f1c839bf7ec7575/src/test/guardian/pocs/IncorrectPriceDecimals.sol#L15>

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/99>

HIGH **H-06**

RESOLVED

Heart Beat DoS'ed On Zero Emissions

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/master/src/policies/deposits/ConvertibleDepositAuctioneer.sol#L605
REVIEW	Main Review

DESCRIPTION

The `heart` beat executes period tasks, including the `EmissionManager.execute`, where the auction parameters are set.

In case of a low premium (or $< 100\%$, which is the current case based on price and backing per ohm), the `emission` will be zero. Therefore, both `target` and `tickSize` params are zero during `setAuctionParameters`, values not allowed in the auctioneer.

Two main issues here:

- `heart.beat()` reverts, as periodic task execution reverts if one of the tasks fails.
- In case there is a live auction and premium drops for the next execution, ticks won't be updated and auction results not stored.

RECOMMENDATION

Make sure periodic tasks do not revert during their execution, preventing complete DoS on the heart beat.

Additionally, due to the fact that emissions are zero, auctioneer should be disabled or paused to prevent more bids. Make sure day state and auction results are updated accordingly.

PROOF OF CONCEPT <https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/guardian/src/test/guardian/pocs/HeartBeatDos.sol>

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/104>

HIGH **H-07**

RESOLVED

User Can Game Rate Received From Vault

CATEGORY Gaming
LOCATION YieldDepositFacility.sol
REVIEW Main Review

DESCRIPTION

In the `claimYield` function for expired positions, users can provide a `timestampHint_` to select a historical vault conversion rate from `vaultRateSnapshots`. The only validation is that the `hint ≤ expiry`, with no check for proximity to expiry or against drops in rate. If a vault's rate peaks mid-period and then falls, users can hint to the peak, inflating yield calculations beyond the actual value at expiry. Consequently, more assets are withdrawn from the vault than expected, which effectively steals the gain of other users.

Consider the following scenario:

- (1) Alice creates a position, the vault accrues yield and a snapshot is taken at time X.
- (2) The strategy loses funds, the share value of the vault decreases, and a snapshot is taken at (X + 8) hours.
- (3) Bob then creates a position and a snapshot is taken at (X + 16) hours.
- (4) Time passes so Bob's position is now past expiry.
- (5) Bob claims yield with a hint at time X, which is before the he even made a position, effectively stealing any yield Alice earned in her duration during the vault.

RECOMMENDATION

Enforce `timestampHint_` is as close to the expiry as possible.

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/d9fabacaefc3fe65d7c64035cedd2f33>

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/84>

HIGH **H-08**

ACKNOWLEDGED

Insolvency Due To Fixed-Amount Withdrawals

CATEGORY	Logical Error
LOCATION	olympus-v3/src/bases/BaseAssetManager.sol at 08e562e6d4e4f3ef7bdf5553de99574403cfdcb9 · OlympusDAO/olympus-v3
REVIEW	Main Review

DESCRIPTION

Olympus intends to support vaults that can realize losses — in other words, vaults where the value of a user's deposit may decrease after the time of deposit.

However, the current code is not equipped to handle such scenarios.

Example – Base Asset Manager

- A user deposits x amount into the vault. Olympus records the deposited asset amount and mints recipient tokens equivalent to that amount.
- When the user withdraws, Olympus allows them to redeem shares equivalent to their initial deposit amount.
- If the vault has incurred losses, this logic allows the user to withdraw more than their proportional share of the vault's current value. This could lead to a bank run and insolvent positions.

RECOMMENDATION

We expect that there may be multiple areas in the code where loss-incurring vaults can cause similar issues. Addressing this in a single place may not be sufficient. A broader review and multiple code changes are likely required to ensure proper handling of loss scenarios.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

RESOLVED

Griefing Receipt Token Conversion

CATEGORY Access Control

LOCATION <https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/master/src/libraries/ERC6909Wrappable.sol#L167>

REVIEW Main Review

DESCRIPTION

In order to convert a receipt token into OHM, users first need to approve the `DepositManager` to spend the tokens, enforced during `_burn`. This approval is also needed for `wrap` or `unwrap` tokens from `ERC6909` to `ERC20`.

However, malicious users can grief the `convert` transaction by front running and wrapping/unwrapping receipt tokens. The `convert` will now fail with insufficient allowance as it was spent by the malicious actor. This is possible due to the fact that `wrap` and `unwrap` do not have access control, and allow any user to perform the action on behalf of other, as long as the allowance was given. The most impact occurs when the attacker repeats this process, until the user's position expires,

In case of a smart contract user, deployed with max approval to `DepositManager` to save gas, receipt tokens can be permanently locked if the ERC6909 tokens are suddenly wrapped into ERC20, without a way to handle these tokens or approval functionality.

RECOMMENDATION

A combination of better documentation on approvals and stricter access control for wrap/unwrap functionality is recommended to mitigate this risk.

Alternatively, consider using `msg.sender` instead of `onBehalfOf` for the `wrap` and `unwrap` functions.

PROOF OF CONCEPT <https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/guardian/src/test/guardian/pocs/GriefReceiptConversions.sol>

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/89>

MEDIUM **M-02**

RESOLVED

Withdraw Rounding Prevents Yield Claims

CATEGORY Logical Error
LOCATION DepositManager.sol
REVIEW Main Review

DESCRIPTION

When claiming yield, the assets are withdrawn from the vault through the deposit manager: `(, uint256 actualAmount) = _withdrawAsset(asset_, recipient_, amount_);`

ERC4626 `withdraw` ends up withdrawing the exact number of assets requested, but it also rounds up the number of shares that are burnt. Because more shares are burnt, `depositedSharesInAssets` is decreased slightly more than expected, making it more likely that the vault is insolvent:

`_assetLiabilities[assetLiabilitiesKey] > depositedSharesInAssets + borrowedAmount`. Ultimately a user attempts to claim yield but is prevented by as little as 1 wei of insolvency due to the extra drop in `depositedSharesInAssets`.

This issue also occurs with `borrowAgainstRedemption` as it triggers `withdraw` as well, and the deficit will continue to compound as more users submit borrows.

RECOMMENDATION

Consider adjusting the withdrawal mechanism such that the user's requested amount to withdraw is rounded down with existing ERC4626 functionality. The user may now receive less than than requested for redemption and this must be appropriately documented (redemptions are not exactly 1:1 anymore).

PROOF OF CONCEPT <https://github.com/GuardianOrg/olympus-v3-olympus-fuzz/pull/4>

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/91>

MEDIUM **M-03**

ACKNOWLEDGED

Emissions Not Adjusted

CATEGORY Logical Error
LOCATION EmissionManager.sol
REVIEW Main Review

— DESCRIPTION

The EmissionManager calculates emissions based on supply (`getSupply() = gohm.totalSupply() * gohm.index() / 10 ** _gohmDecimals`) and premium, assuming a single reserve token. If multiple EmissionManagers are deployed for different reserves (e.g. USDC through separate auctioneers), each independently computes and mints OHM emissions without aggregating across all managers. This duplicates emissions, as `getNextEmission()` in one manager doesn't account for emissions from others, over-minting OHM and diluting supply. For example 2 EmissionManagers/Auctioneers can double the expected OHM emission.

— RECOMMENDATION

Consider accounting for multiple managers within `getNextEmission`. Otherwise, utilize only one EmissionManager and Auctioneer at a time.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

ACKNOWLEDGED

Keepers Not Incentivized To Quickly Beat

CATEGORY Warning
LOCATION Heart.sol
REVIEW Main Review

DESCRIPTION

Within the Heart, the `currentReward()` function pays 0 at the exact beat boundary (current time == lastBeat + frequency()), then ramps linearly to `maxReward` over `min(auctionDuration, frequency())`. Since `beat()` reverts before the boundary and is valid at the boundary with 0 reward, rational keepers are incentivized to wait past the boundary to earn a positive payout. Especially if transaction gas fees are higher, it only makes sense for keepers to wait to get a positive reward that exceeds the gas fee. This leads to execution delay, delayed price moving average updates, etc.

RECOMMENDATION

Consider a base reward at the valid time boundary or run protocol keepers.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-05**

ACKNOWLEDGED

Yield Lost For Expired Positions

CATEGORY	Unexpected Behavior
LOCATION	https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/master/src/policies/deposits/YieldDepositFacility.sol#L326
REVIEW	Main Review

— DESCRIPTION

Creating positions in `YieldDepositFacility` gives user the ability to claim yield from the vault, until the position expires. Once expired, users will only be able to claim up to the expiring timestamp.

These expired positions will still have assets deposited in the vault, accruing yield. However, the yield generated is not distributed to existing users, even after expired positions redeem or reclaim (exit).

This pending yield will be left in the `DepositManager` even after all facility positions exit, with no way to claim it. `YieldDepositFacility` will end up in an invalid state:

- operator liabilities = 0
- shares in assets > 0
- max claim yield > 0

— RECOMMENDATION

Consider adding an admin function to claim yield during emergencies or when operator liabilities becomes 0 (no user deposits and available yield to claim).

PROOF OF CONCEPT <https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/guardian/src/test/guardian/pocs/YieldLostExpiredPosition.sol>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-06**

ACKNOWLEDGED

Converted Reserves Do Not Increase Backing

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/master/src/policies/deposits/ConvertibleDepositFacility.sol#L289
REVIEW	Main Review

— DESCRIPTION

Teller will make a `callback` to `EmissionManager` in order to update the backing price based on new supply and reserves added.

On the other side, the `ConvertibleDepositFacility.convert` function will mint OHM to user, after withdrawing reserves from the vault into the treasury. These reserves will later be deposited into the vault during a periodic task execution in `ReserveWrapper`.

However, this convert flow does not update the backing price before depositing the received reserves and minting the output amount of OHM, compared to teller purchases during `callback` execution.

— RECOMMENDATION

Consider updating backing OHM price when converting notes in the `ConvertibleDepositFacility`.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-07**

ACKNOWLEDGED

First Depositor Attack Affects Redemptions

CATEGORY	Logical Error
LOCATION	DepositManager.sol
REVIEW	Main Review

— DESCRIPTION

Users interacting with a facility are potentially vulnerable to the first depositor inflation attack depending on the underlying vault.

Consider a vault that allows for donations (using `balanceOf`):

- Bob deposits 1 wei USDS into CDF and gets 1 wei rUSDS
- Alice deposits 20,000 USDS into CDF, expecting 20,000 rUSDS
- Bob frontruns and transfers 10,000 USDS directly to the vault
- Alice is minted 15,000 rUSDS but only 1 wei of shares is minted for the DepositManager for the CDF's operator shares
- CDF operator shares is only 2 wei now
- Alice redeems 15,000 rUSDS and withdraws 15,000 USDS, and shares for the CDF operator goes from 2 wei to 1 wei.
- Bob redeems his 1 wei USDS and shares is now 0, but the Vault contains 15,000 USDS.
- Bob deposits 15,000 USDS again, receiving 30,000 rUSDS due to the vault's sitting 15,000 USDS
- Bob redeems 30,000 rUSDS, withdrawing 30,000 USDS, recovering the vault's leftover assets.
- Bob invested 1 wei + 10,000 USDS (donation) + 15,000 USDS (final deposit) and walked away with 30,000 USDS, making 5,000 USDS in profit while Alice lost 5,000 USDS
- Consequently, the `actualAmount` receipt token setup does not prevent the first depositor inflation attack but adds extra steps necessary to execute it.

Note that the severity is of course subject to the length of redemption periods and whether the underlying vault is susceptible to the typical first depositor inflation attack.

— RECOMMENDATION

Ensure the underlying vault uses the OZ virtual offset or dead shares are minted initially.

M-07 First Depositor Attack Affects Redemptions

CONTINUED

PROOF OF CONCEPT <https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/9eb8563bd28a2d76517bf1241aabb974ba0b0e82/src/test/guardian/pocs/FirstDepositor.sol#L18>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-08**

RESOLVED

Capacity Can Grow Incorrectly

CATEGORY Unexpected Behavior
LOCATION ConvertibleDepositAuctioneer.sol
REVIEW Main Review

DESCRIPTION

The `_getCurrentTick` function computes the `capacityToAdd` amount accounting for the number of enabled deposit periods such that the configured aggregate auction target is met per day in capacity additions.

However in the `enableDepositPeriod` and `disableDepositPeriod` functions there is no update to ensure that the latest capacity changes have been reflected up to the current timestamp.

This allows for technically incorrect capacity changes to occur around the enabling and disabling of deposit period.

For example, consider the following scenario:

- There are 2 deposit periods, A & B, enabled for the auctioneer
- The daily target for all auctions in aggregate is 100 tokens
- it has been 3 hours since the last capacity update
- $3 * 100 / 24 = 12.5$ total capacity which should be added to auctions A & B through an increase of 6.25 capacity to both
- A third deposit period is enabled, C
- A deposit occurs for deposit period A right after the deposit period C is enabled
- Now the 12.5 total capacity which has accrued is split amongst deposit periods A, B, and C — giving A only a 4.167 token allocation when it should have received 6.25

RECOMMENDATION

In the `enableDepositPeriod` and `disableDepositPeriod` functions consider updating the capacities for the existing deposit periods with the `_updateTicks` function before a new period is added or removed to reflect the state of the capacities of each deposit period at that time.

M-08 Capacity Can Grow Incorrectly

CONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/96>

MEDIUM **M-09**

RESOLVED

Capacity Calculation Incorrect

CATEGORY	Logical Error
LOCATION	https://github.com/0lympusDAO/olympus-v3/blob/08e562e6d4e4f3ef7bdf5553de99574403cfdcb9/src/policies/deposits/ConvertibleDepositAuctioneer.sol#L397-L398
REVIEW	Main Review

DESCRIPTION

```
uint256 capacityToAdd = (_auctionParameters.target * timePassed) /  
1 days /  
_depositPeriodsCount;
```

This formula assumes all deposit periods are active from the beginning. If a deposit period is enabled or disabled mid-auction:

- Enabling a new period causes under-selling (less OHM minted).
- Disabling a period causes overselling (more OHM minted).

RECOMMENDATION

Restrict enabling/disabling deposit periods to parameter updates (`setAuctionParams`) before tick recalculations. Only allow emergency toggling in exceptional cases.

RESOLUTION

Status: Resolved.

<https://github.com/0lympusDAO/olympus-v3/pull/114>

MEDIUM **M-10**

RESOLVED

Target Update Without LastUpdate Adjustment

CATEGORY Logical Error
LOCATION ConvertibleDepositAuctioneer.sol#L723-L724
REVIEW Main Review

DESCRIPTION

When `setAuctionParams` is called to set a new target, `lastUpdate` is not updated. The new target applies retroactively from the last update, affecting both past and future periods, which may result in artificially discounted or inflated prices.

RECOMMENDATION

Update `lastUpdate` within `setAuctionParams` by setting `setLastUpdate_ = true`.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/112>

MEDIUM **M-11**

RESOLVED

Inconsistent Global VS Per-Period Auction Parameters

CATEGORY	Logical Error
LOCATION	ConvertibleDepositAuctioneer.sol#L291
REVIEW	Main Review

DESCRIPTION

The auction design maintains global variables for:

- `tickSize`
- `target`
- `dayState`

But maintains per-period state for:

- `depositPeriodPreviousTicks[depositPeriod]` (price, capacity, lastUpdate).

This creates asymmetry:

- When tick size is reduced globally (after hitting the daily limit), all deposit periods inherit the smaller tick size.
- However, each deposit period's `previousTick` (price, capacity, lastUpdate) is only updated when a bid occurs in that specific period.

If a new bid occurs in a different deposit period right after tick size reduction, the global tick size reduction applies instantly, but the local `previousTick` state for that period is stale. This leads to:

- `getCurrentTick` running for significantly more iterations (capacity ÷ smaller tick size, plus longer `timePassed`), pushing the price downward excessively.
- `previewBid` also using the reduced tick size, so while the final loop ramps the price upward, the first tick capacity is sold at a direct discount, and later ones start going up from the outdated starting price, resulting in total discount.

Example Scenario

- Two deposit periods (A and B).
- Period A hits its daily limit → tick size halved globally.
- Period B has not been updated since earlier in the day (large capacity, stale price, older lastUpdate).

M-11 Inconsistent Global VS Per-Period Auction Parameters

CONTINUED

- A user bids in Period B:
- Loop runs longer (capacity ÷ smaller tick size).
- Starting price is based on the outdated `previousTick`, which is lower.
- Price steps down further with each iteration, creating unintended discounts.

Net effect: the bidder in Period B acquires OHM cheaper than intended.

— RECOMMENDATION

- Revisit the architecture:
- Either make all auction parameters (tick size, targets, day state) per-period, OR
- Keep them global but synchronize state updates across all deposit periods whenever tick size is reduced.

— RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/113>

LOW **L-01**

RESOLVED

Risk Of Underflow With Multiple Operators

CATEGORY	Warning
LOCATION	BaseDepositFacility.sol
REVIEW	Main Review

— DESCRIPTION

Because function `handleBorrow` does not decrement the appropriate `_assetOperatorCommittedDeposits`, if there are multiple authorized operators for a facility a malicious operator can prevent another operator from withdrawing entirely.

Consider the following scenario:

Operator A commits 50 tokens and Operator B commits 50 tokens, total tokens committed is 100

-> `_assetOperatorCommittedDeposits[A]` reads 50
-> `_assetOperatorCommittedDeposits[B]` reads 50
-> `_assetCommittedDeposits` reads 100

Operator A borrows 50 tokens (`handleBorrow`)

-> `_assetOperatorCommittedDeposits[A]` still reads 50
-> `_assetOperatorCommittedDeposits[B]` reads 50
-> `_assetCommittedDeposits` reads 50

Operator A can now `handleCommitWithdraw`

-> `_assetOperatorCommittedDeposits[A]` reads 0
-> `_assetOperatorCommittedDeposits[B]` reads 50
-> `_assetCommittedDeposits` reads 0

Operator B cannot withdraw anything since `_assetCommittedDeposits` is 0 and underflow reverts on any operation.

Note that for this attack vector to be performed there have to be multiple operators, however only the `DepositRedemptionVault` appears to be a valid operator currently.

— RECOMMENDATION

Update `assetOperatorCommittedDeposits` accordingly when borrowing.

L-01 Risk Of Underflow With Multiple Operators

CONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/94>

LOW **L-02**

RESOLVED

Insolvency When Large Yield

CATEGORY	Logical Error
LOCATION	https://github.com/0lympusDAO/olympus-v3/blob/08e562e6d4e4f3ef7bdf5553de99574403cfdc9/src/policies/deposits/ConvertibleDepositFacility.sol#L358
REVIEW	Main Review

DESCRIPTION

In `DepositManager.claimYield`, withdrawing a large yield relative to a small deposit burns all operator shares via `vault.withdraw`, setting `depositedSharesInAssets` to 0 while liabilities remain. This triggers `DepositManager_Insolvent`, blocking subsequent yield claims.

Consider the following scenario:

- Deposit: 100 assets, 100 shares, 100 receipts.
- Yield: +1e6 assets, vault = 1000100 assets, 100 shares, rate = 10001.0.
- Claim: Withdraws 999999 assets, burns all 100 shares.
- Post-claim: borrowed = 0, shares = 0, depositedSharesInAssets = 0, liabilities = 100 hence $100 > 0 + 0$ which triggers `DepositManager_Insolvent` on claim.

RECOMMENDATION

Consider enforcing a minimum deposit amount as this scenario is more prevalent for small deposits.

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/1d797716a875129fdcaa56fcfcc9e227>

RESOLUTION

Status: Resolved.

<https://github.com/0lympusDAO/olympus-v3/pull/115>

LOW **L-03**

ACKNOWLEDGED

Protocol Risk When Interest Exceeds Buffer

CATEGORY	Warning
LOCATION	DepositRedemptionVault.sol
REVIEW	Main Review

— DESCRIPTION

In DepositRedemptionVault, the loan interest is not capped to ensure it remains below the buffer `(redemption.amount - loan.initialPrincipal)`. Governance can set `_assetAnnualInterestRates` and `_assetMaxBorrowPercentages` such that interest exceeds the buffer, incentivizing users to default rather than repay or extend loans.

Consider the following scenario:

- start redemption 100
- borrow 90 (principal:90 interest:45)
- if you repay => pay 45, claim 100, net = $100 - 45 = 55$
- if you default => do not pay anything, lose everything, but walk away with 90

— RECOMMENDATION

Clearly document and ensure the interest rate and max borrow is set appropriately so that the amount committed is greater.

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

User Can Delay Reclaims

CATEGORY	Censoring
LOCATION	DepositRedemptionVault.sol
REVIEW	Main Review

DESCRIPTION

When a user starts a redemption their assets are committed and the available deposits (`getAvailableDeposits`) is reduced by that amount.

The smaller the available deposits, the less users can `reclaimFor` due to `_validateAvailableDeposits` validation: `if (amount_ > availableDeposits) revert DepositFacility_InsufficientDeposits(amount_, availableDeposits);`

Because there is no penalty for starting a redemption and then cancelling (`cancelRedemption`) to receive their committed tokens back, malicious users can continuously commit their receipt tokens to delay user reclaims.

Consider the following example:

- User A has 100 receipt tokens, deposited 100 tokens
- User B has 100 receipt tokens, deposited 100 tokens
- User A commits 100 to start redemption
- $\text{getAvailableDeposits} = 100$ (shares in assets assume 1:0.5 lossy vault) - 100 = 0
- User B cannot reclaim until User B cancels their redemption

RECOMMENDATION

Consider adding a fee for cancellation.

RESOLUTION

Status: Acknowledged.

LOW **L-05**

RESOLVED

Default VS Reclaim Gaming

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

Depending on the configured parameters, it may be more favorable for a user to borrow, wait the loan's term, and self-default rather than reclaim. Consequently, the protocol treasury's earning may potentially be negatively impacted with the user incentives.

— RECOMMENDATION

Document and take this into consideration when setting protocol parameters.

— RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/93>

LOW **L-06**

RESOLVED

Delayed Heartbeat Affects Auction Tracking

CATEGORY Warning
LOCATION Global
REVIEW Main Review

DESCRIPTION

Everyday the auction stores the convertible OHM in the `_dayState`, and this is recorded within the `_auctionResults`. The expectation is that the EmissionManager will trigger the auction every 24 hours, and every 24 hours there will be an accurate representation of convertible amount to decide whether a bond market needs to be created.

The Heart is not required to beat every 8 hours, so waiting for 3 beats may take longer than 24 hours if there are outages, block congestion, etc. In this case, more convertible amount may be attributed to a day than was actually sold in that day, preventing the creation of the bond market even if there was underselling.

RECOMMENDATION

Consider documenting this behavior.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/105>

LOW **L-07**

RESOLVED

Price Decays Faster Than Expected

CATEGORY	Unexpected Behavior
LOCATION	ConvertibleDepositAuctioneer.sol
REVIEW	Main Review

— DESCRIPTION

In the `ConvertibleDepositAuctioneer` contract the size of each tick decays as the daily expected volume of purchases is surpassed by an incremental multiple.

This allows the price of the auction to move up faster as more convertible deposits are purchased.

However it also allows the price to decay more rapidly due to the logic inside of `_getCurrentTick` which decays the price in correspondence with the number of ticks which would be crossed by the `capacityToAdd` which is added over time.

The outcome of this behavior is that the auction simply allows users to purchase more tokens at a lower average price within a day than may be expected when there is a large volume purchased, which pushes the amount to a multiple of the daily allotment, at the beginning of the day.

— RECOMMENDATION

Be aware of this unexpected increase in the rate at which the auction price can drop after a large volume has been purchased.

To negate this effect, use the initial tick size rather than the `_currentTickSize` in the decay `while` loop in the `_getCurrentTick` function.

— RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/97>

LOW **L-08**

RESOLVED

Deposit Period Enabled Before Policy

CATEGORY	Warning
LOCATION	ConvertibleDepositAuctioneer.sol
REVIEW	Main Review

DESCRIPTION

Enabling a deposit period before the Auctioneer policy is initialized through `_enable` sets the previous tick's price and capacity to 0, as `_auctionParameters.minPrice` and `_auctionParameters.tickSize` are uninitialized. This invalid state persists until re-enabled or updated.

RECOMMENDATION

Add a check in `enableDepositPeriod` to revert if policy is not enabled or clearly document this behavior.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/103>

LOW **L-09**

RESOLVED

Re-enabling Loses Tick Data

CATEGORY Warning
LOCATION ConvertibleDepositAuctioneer.sol
REVIEW Main Review

DESCRIPTION

Disabling and re-enabling a deposit period with functions `disableDepositPeriod` and `enableDepositPeriod` resets the previous tick to the auction's `minPrice` and `tickSize`, losing prior capacity and price data. New capacity from time passed during disablement is not added, leading to lost OHM conversion potential.

RECOMMENDATION

Be aware and clearly document this behavior.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDA0/olympus-v3/pull/103>

LOW **L-10**

RESOLVED

Setting Tracking Period Wipes Existing Results

CATEGORY Warning
LOCATION ConvertibleDepositAuctioneer.sol
REVIEW Main Review

DESCRIPTION

Calling `setAuctionTrackingPeriod` resets `_auctionResults` to a new array of length `days_`, wiping prior results and setting `_auctionResultsNextIndex=0`. This loses historical data, potentially misinforming EmissionManager on underselling and triggering unnecessary bond markets. Note that the number of days can even be set to the same amount, and the historical data will be wiped.

RECOMMENDATION

Be aware and clearly document this behavior.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDA0/olympus-v3/pull/103>

LOW **L-11**

RESOLVED

Failed Bond Market Creation Derails Auction

CATEGORY Warning
LOCATION EmissionManager.sol
REVIEW Main Review

DESCRIPTION

The EmissionManager tightly couples auction parameter updates with bond market creation. If `bondAuctioneer.createMarket` fails (e.g. `allowNewMarkets = false`), the entire emission execution reverts. The auction will continue to operate with stale parameters -- minimum auction price, current tick size will not be adjusted to reflect latest emission rate and OHM price. Furthermore, the auction results will not be stored for the day.

RECOMMENDATION

Consider having governance safety mechanisms in-place to set auction parameters and data in the case of failure.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/105>

LOW **L-12**

ACKNOWLEDGED

Disabling Policy Loses Auction Results

CATEGORY Warning
LOCATION ConvertibleDepositAuctioneer.sol
REVIEW Main Review

DESCRIPTION

Disabling the Auctioneer before the 3rd Heartbeat (24 hours) skips `_storeAuctionResults` in `setAuctionParameters`, losing the daily convertible data. Re-enabling the policy starts the tracking from scratch, potentially triggering unnecessary bond markets due to perceived underselling.

RECOMMENDATION

Be aware and clearly document this behavior.

RESOLUTION

Status: Acknowledged.

LOW **L-13**

ACKNOWLEDGED

Allowance Decrease Frontrunning

CATEGORY	Warning
LOCATION	CloneERC20.sol
REVIEW	Main Review

DESCRIPTION

The `approve` function simply resets the allowance to the new value: `allowance[msg.sender][spender] = amount;`

This allows a spender to utilize the prior approval before the new one is set. For example, if the spender has a previous allowance of 100e18 and the owner wants to decrease this to 10e18, the spender can frontrun this call and consume the full 100e18 allowance, while getting an additional allowance of 10e18 granted.

RECOMMENDATION

Clearly document this risk with the `approve` function.

RESOLUTION

Status: Acknowledged.

LOW **L-14**

RESOLVED

Task Execution Succeeds

CATEGORY	Warning
LOCATION	BasePeriodicTaskManager.sol
REVIEW	Main Review

— DESCRIPTION

A low-level call is used to trigger a custom selector on a periodic task address. If the address does not have contract code, the call will return with `success = true` even if no task was truly executed.

— RECOMMENDATION

Ensure only valid contract addresses are added for tasks.

— RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/101>

LOW **L-15**

RESOLVED

Max Yield Cannot Be Claimed

CATEGORY	Warning
LOCATION	YieldDepositFacility.sol
REVIEW	Main Review

DESCRIPTION

The `maxClaimYield` function aims to "return the maximum yield that can be claimed for an asset and operator pair", but it does not represent the true amount that can be claimed from within the YieldDepositFacility. A portion of the yield will be unclaimable through the YDF because the last snapshot rate for a position is adjusted by 1 wei, as well as precision loss when calculating a position's current value. Therefore, the calculated yield across multiple positions and users will not sum up to the `maxYield` returns for the YDF operator. This may be unexpected for integrators and off-chain systems relying on `maxClaimYield`.

RECOMMENDATION

Clearly document that `maxClaimYield` is the theoretical max and may not be reached by all users claiming their current yield.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/109>

LOW **L-16**

RESOLVED

Cross-Contract Reentrancy Risk

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

Throughout the contracts, `safeTransferFrom` is not performed at the very top of the function. This opens up attack vectors for ERC777 deposit tokens to trigger a cross-contract reentrancy since state updates are performed before the tokens are received.

— RECOMMENDATION

Move `safeTransferFrom` to the top of functions and take into consideration which tokens are allowed within the protocol.

— RESOLUTION

Status: Resolved.

<https://github.com/0lympusDA0/olympus-v3/pull/102>

LOW **L-17**

RESOLVED

Parameters Should Be Configurable

CATEGORY	Warning
LOCATION	DepositRedemptionVault.sol
REVIEW	Main Review

— DESCRIPTION

Parameters such as `_assetMaxBorrowPercentages` are dependent solely on the deposit token, and are the same across all facilities. However, some facilities may have different risk profiles, hence the borrow percentages, reclaim rates, etc. should be adjustable according to the facility as well.

— RECOMMENDATION

Consider allowing for more granularity in parameter adjustments by also accounting for the facility.

— RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/111>

LOW **L-18**

RESOLVED

Lack Of Validation On Withdrawal

CATEGORY Warning
LOCATION BaseAssetManager.sol: 114
REVIEW Main Review

DESCRIPTION

There are multiple entrypoints in the DepositManager that access function `_withdrawAsset` such as `claimYield`, `borrowingWithdraw`, and `withdraw`.

Currently only `claimYield` validates that the DepositManager is solvent, but it would be advisable that `_withdrawAsset` itself would have the validation as a chokepoint pattern to ensure all withdrawals leave the DepositManager solvent and if there are any issues the Olympus team is swiftly aware.

RECOMMENDATION

Consider adding solvency validation to `_withdrawAsset`.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/107>

LOW **L-19**

RESOLVED

Missing Validation For previewReclaim

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/master/src/policies/deposits/BaseDepositFacility.sol#L300
REVIEW	Main Review

DESCRIPTION

During reclaims, the amount being withdrawn is compared to the available deposits, accounting for commitments, to verify if there are enough assets to reclaim.

However, deposit manager will only allow to withdraw up to the asset liabilities of the given facility, which only increase during deposits. Given the fact that YDF users can reclaim using CDF, withdrawing funds may underflow as the amount was not validated against the facility liabilities:

```
_assetLiabilities[_getAssetLiabilitiesKey(params_.asset, msg.sender)] -= params_.amount;
```

RECOMMENDATION

During `previewReclaim`, verify that the `amount` param is lower or equal to the facility liabilities in the `DepositManager`, not the available deposits (which include yield).

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDA0/olympus-v3/pull/108>

LOW **L-20**

RESOLVED

Initial Auction Parameters Not Validated

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/master/src/policies/deposits/ConvertibleDepositAuctioneer.sol#L605
REVIEW	Main Review

DESCRIPTION

The `EmissionManager` is responsible for periodically tuning the auction parameters, which are calculated based on the emissions and current OHM price. The `tickSize_` is calculated as a % of the `target_`.

However, during the `ConvertibleDepositAuctioneer` policy enable flow, there is no validation on `tickSize_ < target_`. If the tick size is greater than the target, it will open unexpected behaviors, as the auctioneer will decrease the tick size faster due to higher values of the multiplier.

RECOMMENDATION

Consider not only validating non zero values for the auction params, but ensuring `tickSize_ < target_`, in line with the `EmissionManager`. This can be done in the `_enable` as the issue only appears during policy enabling.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/106>

LOW **L-21**

ACKNOWLEDGED

Risk-Free Bids

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

Based on the current Auction status, users can bid for convertible notes and ideally purchase a position with the lowest conversion price to make a profit from longing OHM.

However, even if price does decrease, users can still start a redemption and receive their entire deposit back 1:1. Consequently, users have no direct asset-risk for utilizing convertible notes. Olympus should confirm this is intended mechanics for convertible notes, especially since most auctions will begin at an OHM price below market, effectively creating a risk-free money printer for some.

— RECOMMENDATION

Consider if this is intended behavior. If so, consider extra fees upon redemptions or bids. Furthermore, consider implementing the possibility for a time delay before conversion.

— RESOLUTION

Status: Acknowledged.

LOW **L-22**

RESOLVED

Asymmetric Yield Across Multiple Claims

CATEGORY Gaming
LOCATION YieldDepositFacility.sol: 333
REVIEW Main Review

DESCRIPTION

Function `claimYield` updates the position's last snapshot rate after each yield claim, setting the `lastSnapshotRate` to a higher value for subsequent calculations as yield is earned. This increases the denominator in the `lastShares` computation for remaining growth periods, resulting in fewer effective shares (`lastShares`) and lower total yield when claiming multiple times compared to a single claim over the same period and end rate. Ultimately, users who claim yield multiple times during a position's period will receive less total yield than those who claim once.

RECOMMENDATION

Clearly document this behavior to users.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/84>

LOW **L-23**

ACKNOWLEDGED

ERC-4626 Deposit VS PreviewRedeem Invariant

CATEGORY	Superfluous Code
LOCATION	BaseAssetManager.sol#L90-L91
REVIEW	Main Review

— DESCRIPTION

In yield deposit facilities, Olympus first deposits tokens into the vault and then calls `previewRedeem` to determine the owed assets, minting equivalent receipt tokens.

However, there is no guarantee that, `previewRedeem(deposit(amount x)) ≈ x` would always holds true for all kinds of vaults.

This may deviate if the vault charges withdrawal fees or considers available liquidity when calculating redemptions.

— RECOMMENDATION

Before onboarding any vault, review its behavior to ensure this invariant holds. Specifically, confirm that `previewRedeem(deposit(x)) ≈ x` for the chosen vault, otherwise deposits may miscalculate user entitlements.

— RESOLUTION

Status: Acknowledged.

LOW **L-24**

ACKNOWLEDGED

Vault Validation Warnings

CATEGORY	Validation
LOCATION	DepositManager.sol
REVIEW	Main Review

— DESCRIPTION

Each asset has a specified deposit cap (`assetConfiguration.depositCap`) which should not be exceeded when depositing assets through the DepositManager. Each asset also has an arbitrarily configured ERC4626 vault which was set during the `addAsset` function call by a manager or admin.

Because the vault can require a minimum deposit amount which is not currently enforced directly in the DepositManager, there is potential for the minimum deposit to be greater than the amount of capacity leftover before exceeding the `assetConfiguration.depositCap` and users will be unable to deposit even when there is available space.

Another potential asymmetry is if the vault has its own deposit cap, and users may attempt deposits through the DepositManager that will simply fail when the deposit into the vault is attempted and the vault's cap is exceeded, although there is still available space from the DepositManager's perspective.

— RECOMMENDATION

Try to maintain alignment between the vault's configuration and DepositManager's configuration to avoid unexpected behavior and/or document this risk.

— RESOLUTION

Status: Acknowledged.

LOW **L-25**

RESOLVED

Asymmetric Remaining Deposit

CATEGORY Warning
LOCATION Global
REVIEW Main Review

DESCRIPTION

Function `YieldDepositFacility::createPosition` uses `remainingDeposit: params_.amount` but this may slightly differ from the `actualAmount` returned by the deposit due to ERC4626 rounding. This `remainingDeposit` value is asymmetric with `ConvertibleDepositFacility::createPosition` which sets it as `remainingDeposit: actualAmount`. Consequently, the `currentValue` during yield calculations within the `YieldDepositFacility` may be just slightly larger than intended.

RECOMMENDATION

Consider using `remainingDeposit: actualAmount`.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/87>

LOW **L-26**

RESOLVED

Lack Of Slippage Protection On Bids

CATEGORY	MEV
LOCATION	ConvertibleDepositAuctioneer.sol
REVIEW	Main Review

— DESCRIPTION

As users call function `bid()` to purchase convertible deposit tokens, the tick prices will continue to increase as demand grows. However, there is no slippage protection mechanism in the function, such that a user may receive a much lower `output.ohmOut` than expected.

— RECOMMENDATION

Consider adding slippage-protection on the output amount in function `bid()`.

— RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/88>

LOW **L-27**

RESOLVED

Split Griefing Increases Gas Cost

CATEGORY	Gas Griefing
LOCATION	OlympusDepositPositionManager.sol
REVIEW	Main Review

DESCRIPTION

The `split` function in `OlympusDepositPositionManager` allows a position owner to split a position into smaller positions (e.g., 1 wei) and assign them to an arbitrary receiver, appending new position IDs to the receiver's `_userPositions` array. A malicious user can repeatedly split tiny amounts to a recipient, bloating their `_userPositions` array. This increases gas costs for the receiver in `transferFrom`, which loops over `_userPositions` to remove the transferred ID. While testing it was found an attacker would expend more gas than the recipient would lose with the bloated list of positions, making DoS scenarios unlikely.

RECOMMENDATION

Consider enforcing a minimum amount for the `split` function and/or clearly document this risk.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/92>

LOW **L-28**

RESOLVED

Potential Superior Loans

CATEGORY	Configuration
LOCATION	DepositRedemptionVault.sol
REVIEW	Main Review

DESCRIPTION

In the `DepositRedemptionVault`, depending on the interest rate and `_claimDefaultRewardPercentage` a borrower may be able to borrow and pay a lower total amount than compared to the interest rate by defaulting themselves.

For example:

- Consider an interest rate of 12% annually
- A `_assetMaxBorrowPercentages` of 80%
- A `_claimDefaultRewardPercentage` of 50%
- The interest for a 12 month period deposit of 100 tokens is 12 tokens
- The principal for the 12 month period deposit is 80 tokens, leaving 20 as a buffer
- If the user pays off their loan before the end of the 12 month period, they will pay back the 80 token principal + the 12 token interest
- If the user instead waits until the end of their loan period and defaults themselves, they will lose the 20 token buffer, but gain half of that as a default reward

In the case where the user defaults themselves they keep the 80 token borrowed amount and only lose the 10 token half of the buffer, rather than paying the 12 tokens in interest.

RECOMMENDATION

Be aware of this superior loan strategy and consider this when configuring the `_assetMaxBorrowPercentages`, `_claimDefaultRewardPercentage`, and interest rate.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/93>

LOW **L-29**

RESOLVED

Handle Loan Functions

CATEGORY	Validation
LOCATION	BaseDepositFacility.sol
REVIEW	Main Review

DESCRIPTION

The `handleBorrow` and `handleLoanRepay` functions neglect to update the `_assetOperatorCommittedDeposits` mapping values, while they do update the `_assetCommittedDeposits` for the asset, allowing the total committed deposits for an asset to disagree with the sum of all operator committed deposits. Furthermore, this allows one operator to "steal" committed deposits from another.

Consider, `ConvertibleDepositFacility` with Operators A and B

Operator A commits 50 tokens and Operator B commits 50 tokens, total tokens committed is 100

-> `_assetOperatorCommittedDeposits[A]` reads 50

-> `_assetOperatorCommittedDeposits[B]` reads 50

-> `_assetCommittedDeposits` reads 100

Operator A borrows 50 tokens

-> `_assetOperatorCommittedDeposits[A]` still reads 50

-> `_assetOperatorCommittedDeposits[B]` reads 50

-> `_assetCommittedDeposits` reads 50

Operator A can now `handleCommitWithdraw`

-> `_assetOperatorCommittedDeposits[A]` reads 0

-> `_assetOperatorCommittedDeposits[B]` reads 50

-> `_assetCommittedDeposits` reads 0

Operator B cannot withdraw anything since `_assetCommittedDeposits` is 0 and underflow reverts on any operation.

Currently there is no issue related to this as there is only one planned operator, the

`DepositRedemptionVault`, and the logic within the `DepositRedemptionVault` does not allow for this to be exploited.

L-29 Handle Loan FunctionsCONTINUED

— RECOMMENDATION

Consider updating the `_assetOperatorCommittedDeposits` to correspond with the `_assetCommittedDeposits` update in the `handleBorrow` and `handleLoanRepay` functions

— RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/94>

LOW **L-30**

ACKNOWLEDGED

Borrowing More Is Incentivized For Defaults

CATEGORY Warning

LOCATION <https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/03e829da3595f26ee934bc91a6938cd14d409a7a/src/policies/deposits/DepositRedemptionVault.sol#L688>

REVIEW Main Review

DESCRIPTION

Upon loan default, the `retainedCollateral` which serves as the buffer between user collateral value and borrow amount is withheld from the user: `uint256 retainedCollateral = redemption.amount - loan.initialPrincipal`.

Borrowers are incentivized to borrow more if defaulting since the `retainedCollateral` is smaller when the `initialPrincipal` is larger. This may encourage users who do not have intention of paying the interest to maximally borrow funds and reduce available deposits, which may affect features such as reclaims which require available deposits.

This creates a counterintuitive outcome where borrowers taking larger loans are penalized less, incentivizing higher borrowing. Assets with lower borrowing capacity could result in disproportionate losses to users.

RECOMMENDATION

Consider aligning with the standard liquidation practices:

- Keeper receives a cut proportional to borrowed amount.
- Protocol receives a liquidation penalty proportional to borrowed amount.
- Remainder is returned to the position holder.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

RESOLVED

YDF conversionPrice Should Use Constant

CATEGORY Best Practices

LOCATION <https://github.com/0lympusDAO/olympus-v3/blob/08e562e6d4e4f3ef7bdf5553de99574403cfdc9/src/policies/deposits/YieldDepositFacility.sol#L131>

REVIEW Main Review

DESCRIPTION

In `YieldDepositFacility::createPosition`, the `conversionPrice` for a yield-only position is hardcoded to `type(uint256).max`.

The DEPOS module already defines a `constant NON_CONVERSION_PRICE = type(uint256).max` for non-convertible positions. Hardcoding risks misalignment if the module is updated with a different `NON_CONVERSION_PRICE` in the future, even if this is an unlikely scenario.

RECOMMENDATION

Consider using the existing `NON_CONVERSION_PRICE` constant in `YieldDepositFacility::createPosition`.

RESOLUTION

Status: Resolved.

<https://github.com/0lympusDAO/olympus-v3/pull/109>

INFORMATIONAL I-02

RESOLVED

Empty Position Id List Validation Missing

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/08e562e6d4e4f3ef7bdf5553de99574403cfdcb9/src/policies/deposits/ConvertibleDepositFacility.sol#L289>

REVIEW Main Review

DESCRIPTION

The `convert` function in `ConvertibleDepositFacility` does not explicitly check if `positionIds_.length > 0`, allowing calls with empty arrays. While this results in `receiptTokenIn` and `convertedTokenOut` being 0, the transaction reverts safely during `MINTR.mint0hm` (which disallows zero mints). However, this lacks an early revert, potentially wasting gas and reducing user clarity.

RECOMMENDATION

Consider reverting `if (positionIds_.length == 0)` to be symmetrical with `YieldDepositFacility` validations.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/109>

INFORMATIONAL I-03

RESOLVED

depositAmount Calculation Rounds

CATEGORY	Rounding
LOCATION	ConvertibleDepositAuctioneer.sol
REVIEW	Main Review

DESCRIPTION

In the `_previewBid` function the `depositAmount = convertibleAmount.mulDiv(output.tickPrice, _ohmScale)` calculation which occurs when the `depositAmount` must be derived from the total available capacity at the current tick uses round down division.

Therefore the resulting `depositAmount` is rounded down from what the user has to put into the conversion from their total deposit. This however rounds against the favor of the protocol and in favor of the user.

RECOMMENDATION

Out of an abundance of caution, to follow best practices of rounding this calculation should round against the user and estimate a larger `depositAmount` that should be deducted from the user's total deposits using `mulDivUp`.

Notice that this may result in cases where the `remainingDeposit` would be subtracted such that it underflows and causes a panic revert due to additional wei being removed. In this case the `remainingDeposit` value should be minimized to zero.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/109>

INFORMATIONAL **I-04**

RESOLVED

Unwrapped Receipt Necessary For Redemption

CATEGORY	Documentation
LOCATION	DepositRedemptionVault.sol
REVIEW	Main Review

DESCRIPTION

Only unwrapped receipt tokens are supported to start redemptions due to `_pullReceiptToken` requiring the ERC6909 function. Furthermore, function `reclaimFor` requires unwrapped tokens due to `isWrapped: false`. This is safe from a security perspective, but should be clearly documented that users must unwrap before beginning redemptions/reclaiming.

RECOMMENDATION

Clearly document this behavior.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/110>

INFORMATIONAL I-05

RESOLVED

Actual Amount Ignored During Bids

CATEGORY Informational

LOCATION <https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/master/src/policies/deposits/ConvertibleDepositAuctioneer.sol#L235>

REVIEW Main Review

DESCRIPTION

The `YieldDepositFacility.createPosition` returns the `actualAmount` which represents the amount of receipt tokens minted.

Even though the `ConvertibleDepositFacility.createPosition` returns this same parameter, this function is only callable by the `ROLE_AUCTIONEER` which is granted to the `ConvertibleDepositAuctioneer`.

The `ConvertibleDepositAuctioneer.bid` does not return the `actualAmount`. Therefore, integrations will need to rely on the receipt token balance delta to calculate this amount.

RECOMMENDATION

Return the `actualAmount` parameter during bids

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/109>

INFORMATIONAL **I-06**

RESOLVED

Redundant Boolean Check

CATEGORY Best Practices
LOCATION DepositRedemptionVault.sol: 678
REVIEW Main Review

DESCRIPTION

Within function `claimDefaultedLoan`, the expression `loan.isDefaulted == true` is not necessary and can be shorted to `loan.isDefaulted`.

RECOMMENDATION

Consider changing the expression to `loan.isDefaulted`.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/110>

INFORMATIONAL I-07

RESOLVED

Using Magic Numbers

CATEGORY	Best Practices
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

The codebase uses magic numbers to represent certain values. For example:

- `100e2` instead of `ONE_HUNDRED_PERCENT`, used in `_previewBorrowAgainstRedemption`, `setMaxBorrowPercentage`, `setAnnualInterestRate`, `setClaimDefaultRewardPercentage`
- `12` instead of creating `TWELVE_MONTHS` constant
- `30 days` instead of creating `ONE_MONTH` constant

RECOMMENDATION

Consider using constants instead of magic numbers, to avoid mistakes.

RESOLUTION

Status: Resolved.

<https://github.com/0lympusDAO/olympus-v3/pull/91/>

INFORMATIONAL I-08

RESOLVED

Superfluous Permission Request

CATEGORY	Superfluous Code
LOCATION	https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/master/src/policies/deposits/ConvertibleDepositFacility.sol#L79
REVIEW	Main Review

DESCRIPTION

The `ConvertibleDepositFacility` policy has a requests permission for the `MINTR.decreaseMintApproval`, but the contract never calls this function.

RECOMMENDATION

Remove superfluous permission.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/109>

INFORMATIONAL I-09

RESOLVED

Borrows Pay Full Interest Regardless Of Payback

CATEGORY Documentation
LOCATION DepositRedemptionVault.sol
REVIEW Main Review

DESCRIPTION

If a user borrows deposit tokens for a short period of time (e.g. a single block) and proceeds fully repay their loan, the user still owes the entire interest as if they had an active loan for the entire deposit period. This may be unexpected for some users, especially who are accustomed to time based interest mechanisms.

RECOMMENDATION

Clearly document this behavior to users.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/110>

INFORMATIONAL **I-10**

ACKNOWLEDGED

Newly Minted OHM Not Counted In Supply

CATEGORY Warning
LOCATION EmissionManager.sol
REVIEW Main Review

DESCRIPTION

Function `getSupply()` return supply as `(gohm.totalSupply() * gohm.index()) / 10 ** _gohmDecimals;`, but this does not consider any newly minted OHM. Consequently, the emission may be lower than intended.

RECOMMENDATION

Clarify if this is intended behavior.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-11**

ACKNOWLEDGED

Convertible Deposit Facility (CDF)

CATEGORY Logical Error
LOCATION BaseDepositFacility.sol#L314-L315
REVIEW Main Review

DESCRIPTION

Within the Convertible Deposit Facility, both redemptions and conversions can occur against the same position. The user just needs to acquire more receipt tokens through a `deposit` or `createPosition` to have both be in parallel. While no immediate issue was identified, this is a feature/inconsistency protocol team should be aware of.

RECOMMENDATION

Be aware of this duality during future modifications.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-12

RESOLVED

Unnecessary Day State Storage Read

CATEGORY Gas Optimization

LOCATION <https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/master/src/policies/deposits/ConvertibleDepositAuctioneer.sol#L298>

REVIEW Main Review

DESCRIPTION

The `_previewBid` will loop until `remainingDeposit == 0`, increasing the tick price and size.

If there is not enough capacity in the current tick, the size is calculated based on `_getNewTickSize` which uses `_dayState.convertible` to account for daily ohm bids. However, this daily value is constant throughout the while loop, creating unnecessary storage reads.

RECOMMENDATION

Consider caching the `_dayState.convertible` before the while loop, and using the memory variable instead.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/109>

INFORMATIONAL **I-13**

ACKNOWLEDGED

Redundant Setting Of Asset And Period

CATEGORY	Logical Error
LOCATION	ConvertibleDepositFacility.sol#L316-L317
REVIEW	Main Review

— DESCRIPTION

Asset and deposit period values are expected to remain consistent across all entries of a position. Currently, these values are set repeatedly within the loop for each iteration, which is redundant.

— RECOMMENDATION

Optimize by setting `asset` and `periodMonths` once for the first position and reuse them, rather than resetting on each loop iteration.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-14**

ACKNOWLEDGED

Unnecessary While Loop

CATEGORY Gas Optimization
LOCATION ConvertibleDepositAuctioneer.sol: 397
REVIEW Main Review

DESCRIPTION

The `_getCurrentTick` function uses a `while (newCapacity > _currentTickSize)` loop to iteratively reduce capacity and decay the tick price until capacity fits within the tick size or hits `minPrice`. This is unnecessary, as the number of decay steps and final price can be computed directly without looping.

RECOMMENDATION

Consider replacing the loop with a direct calculation.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-15

RESOLVED

Lack Of SafeTransfer

CATEGORY	Best Practices
LOCATION	YieldDepositFacility.sol
REVIEW	Main Review

DESCRIPTION

In the `YieldDepositFacility` contract, the `transfer` function is used when claiming yield, but best practices is to use `safeTransfer` to handle the return values of the various assets that can be configured.

RECOMMENDATION

Consider using `safeTransfer` instead from `SafeERC20`.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/95>

HIGH **H-01**

RESOLVED

Reclaim DoS When Actual Less Than Discounted

CATEGORY Rounding
LOCATION BaseDepositFacility.sol: 410
REVIEW Remediation Review

DESCRIPTION

In function `reclaim`, a discount is calculated on the `amount_` the user requests, and the `amount_` is attempted to be withdrawn from the `DepositManager` to cover the discounted amount as well as the treasury amount.

Because function `DEPOSIT_MANAGER.withdraw` no longer withdraws exactly the `amount_` requested, it is now possible for the `actualAmount` withdrawn to be a couple wei below what was requested.

Consequently, it is now possible for `actualAmount` to be less than `discountedAssetsOut`, which would either cause:

1. `ERC20(address(depositToken_)).safeTransfer(msg.sender, discountedAssetsOut);` to revert in the case that `discountedAssetsOut` is less than the `actualAmount` directly.
2. If the above transfer succeeds, then the next transfer would panic underflow with `actualAmount - discountedAssetsOut`.

Ultimately, the `reclaim` functionality is DoS'd due to the latest `DEPOSIT_MANAGER.withdraw` mechanics because of the rounding changes.

RECOMMENDATION

Consider enforcing that the `discountedAssetsOut` is no more than the `actualAmount` or apply a lower discount rate.

PROOF OF CONCEPT <https://github.com/GuardianOrg/olympus-v3-olympus-fuzz/pull/21>

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/120>

HIGH **H-02**

RESOLVED

Borrowing Insolvency Prevents Borrows

CATEGORY Rounding
LOCATION DepositRedemptionVault.sol
REVIEW Remediation Review

DESCRIPTION

When users borrow using `DepositRedemptionVault`, assets are withdrawn from the vault. Due to the rounding issues when redeeming, the actual amount withdrawn is 1 wei less than the amount requested.

The `_borrowedAmounts` mapping is decreased by this actual amount, instead of the `params_.amount`. Therefore, the remaining shares in assets plus this new borrowed value will be lower than operator liabilities, creating an insolvent case.

RECOMMENDATION

Consider reducing the `_borrowedAmounts` mapping by the requested amount instead of the `actualAmount`. Additionally, user principal should still be equal to the requested amount so the `borrowedAmounts` mapping is not populated with leftover wei after a repay.

PROOF OF CONCEPT <https://github.com/GuardianOrg/olympus-v3-olympus-fuzz/pull/23>

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/123>

HIGH **H-03**

ACKNOWLEDGED

Reclaims Can Force Insolvent Deposit Managers

CATEGORY Logical Error
LOCATION YieldDepositFacility.sol
REVIEW Remediation Review

DESCRIPTION

A user can call function `reclaim` to reclaim their entire deposit, but this does not update the user's DEPOS position, allowing users to claim yield on non-existent assets after reclaiming.

This vector allows for the following scenario:

- (1) Alice deposits $1e18$ reserve tokens through `createPosition`
- (2) Alice then reclaims her entire deposit, receiving a discounted amount. The DepositManager now holds 0 vault shares, but Alice's yield-earning position remains.
- (3) Bob then proceeds to deposit into the vault through `createPosition`.
- (4) Yield accrues and a snapshot is taken.
- (5) Alice claims yield (`claimYield`) with her phantom position, receiving some reserve tokens.
- (6) Bob attempts to claim yield, but because Alice already withdrew some assets, Bob's claim yield attempt reverts with `DepositManager_Insolvent`. Bob cannot claim the yield he has earned with the assets he deposited into the vault and that are generating yield in the vault.

Ultimately, Alice is able to prevent other users from claiming their yield by forcing insolvency through a reclaim, and also claim yield without having actively deposited assets in the vault.

Also note that the smaller the discount is configured (reclaim rate can be as high as 100%), the faster Alice will earn enough yield to overcome any immediate loss.

RECOMMENDATION

Adjust the remaining deposit of existing positions to reflect the withdrawal. Furthermore, ensure the reclaim rate is large enough to discourage malicious behavior.

H-03 Reclaims Can Force Insolvent Deposit Managers

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

HIGH **H-04**

ACKNOWLEDGED

Gas Intensive YDF Prevents Yield Claims

CATEGORY	DoS
LOCATION	YieldDepositFacility.sol
REVIEW	Remediation Review

— DESCRIPTION

To remedy H-07, a Timestamp Linked List was introduced to manage snapshot history and always be able to give a user the latest snapshot rate since expiry. Notably, a snapshot is taken every time a user creates a position which increases the length of the list and necessary gas cost when the next user creates a position.

Furthermore, when the position is past expiry, many timestamps may be added to the list post-expiry, such that more iterations are needed to find the position's latest expiry with function `findLastBefore`.

Although there is a malicious case where attackers create many positions to stuff the list, it is also expected that naturally thousands of positions will be created to the point where:

- (1) `createPosition` becomes too expensive and users do not want to create positions nor use the YDF entirely.
- (2) Existing positions do not claim their yield as the gas costs exceed the yield they gained during their deposit period.

While testing with 10,000 rate timestamp snapshots taken since a position's expiry, a user had to pay ~5,000,000 gas units to create a position and ~3,000,000 gas units to claim yield. With gas prices 4.5 gwei and ETH at \$4,600, gas costs will be \$60 USD -- high enough to overcome the yield from any smaller position.

— RECOMMENDATION

Consider using a Bitmap instead of a Linked List. This would allow for $O(1)$ insertion as well as $O(1)$ search given the proper hint is provided, and that hint that can be stored off-chain.

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/50c52fa8bab39eb72fbec8670f35ffffb>

H-04 Gas Intensive YDF Prevents Yield Claims

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

HIGH **H-05**

RESOLVED

Auctions DoS'd By Increasing Price

CATEGORY	DoS
LOCATION	ConvertibleDepositAuctioneer.sol
REVIEW	Remediation Review

— DESCRIPTION

As reported in H-01, in case of a low premium (or $< 100\%$, which is the current case based on price and backing per ohm), the emission will be zero. When the emission is 0, the tick size is now 1, which is intended to "disable" the auction as prices will drastically increase and quickly with each user bid.

Another key point is that convertible positions are effectively risk-free. If the conversion price is not suitable, a user will just begin a redemption and retrieve 100% of their deposited funds over some time.

This allows for the following, likely attack vector:

- (1) Emission is 0, hence auction is created with tick size of 1 wei
- (2) Alice begins to bid and drastically increase the price
- (3) Auction "re-enabled" with new emission target being non-zero
- (4) Because the price is so high, any genuine bids would output 0 OHM and revert with `ConvertibleDepositAuctioneer_ConvertedAmountZero`
- (5) The Auction is now DoS'd for a prolonged period of time because lots of new capacity is necessary to drop the price to a reasonable level.

— RECOMMENDATION

Instead of arbitrarily setting a small size for 0 emissions, pause the auction process and consider requiring a minimum size for an auction.

PROOF OF CONCEPT <https://github.com/GuardianOrg/olympus-v3-olympus-fuzz/pull/28>

— RESOLUTION

Status: Resolved.

H-05 Auctions DoS'd By Increasing Price

CONTINUED

<https://github.com/OlympusDA0/olympus-v3/pull/119>

MEDIUM **M-01**

ACKNOWLEDGED

Claim Yield Revert With High Yield Fee

CATEGORY Logical Error
LOCATION YieldDepositFacility.sol
REVIEW Remediation Review

DESCRIPTION

The `_yieldFee` within the YDF can be set to near 100%. In such cases, the `yieldFee` can exceed the `actualAmount` leading to a ERC20 insufficient balance revert when transferring the `yieldFee` to the TRSRY.

In the most extreme `_yieldFee` of 100%, the `actualYieldMinusFee` may be zero, and `ERC20(address(asset)).safeTransfer(msg.sender, actualYieldMinusFee);` may revert as some tokens do not allow zero amount transfers.

Ultimately, user may not be able to claim yield when the `yieldFee` is high.

RECOMMENDATION

Consider having the `yieldFee` be based on the `actualAmount`. Otherwise, ensure the `_yieldFee` is low enough to prevent blocked claims. Furthermore, do not perform `ERC20(address(asset)).safeTransfer(msg.sender, actualYieldMinusFee);` unless `actualYieldMinusFee > 0`.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

RESOLVED

Split To Bypass Minimum Deposit

CATEGORY Logical Error
LOCATION BaseDepositFacility.sol
REVIEW Remediation Review

DESCRIPTION

A minimum deposit was introduced to prevent malicious deposits and small, likely malicious positions. However, a user can split their existing position to make the new position's remaining deposit below the minimum deposit.

RECOMMENDATION

Validate minimum deposit on split on the old and new position.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/121>

LOW **L-01**

ACKNOWLEDGED

Period Enabled in Auctioneer Is Disabled In DM

CATEGORY	Validation
LOCATION	Global
REVIEW	Remediation Review

— DESCRIPTION

In the auctioneer it is possible to enable a deposit period with `enableDepositPeriod` that is disabled through the DepositManager and vice versa. Consequently, it is also possible for a user to initially `bid` and create a position when the period is enabled in both, and afterwards when disabled in the DepositManager the user is unable to `convert`.

— RECOMMENDATION

Try to maintain symmetry between enabled deposit periods in the Auctioneer and the DepositManager.

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Lack Of Access Control On Create Token

CATEGORY Access Control
LOCATION ReceiptTokenManager.sol
REVIEW Remediation Review

DESCRIPTION

Function `ReceiptManager.createToken` does not have any access control. An arbitrary user can call `createToken` and bloat space with receipt tokens that have no connection to the DepositManager.

RECOMMENDATION

Make `createToken` a permissioned function.

RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Minimum Deposit and Caps Not Granular

CATEGORY	Best Practices
LOCATION	BaseAssetManager.sol
REVIEW	Remediation Review

DESCRIPTION

Currently the `minimumDeposit` and `depositCap` are configured directly on the asset in the `assetConfiguration` mapping, meaning all periods/facilities/etc. share the same minimum deposit and deposit cap.

It may be prudent to allow more granular parameters to account for different risks across facilities rather than on the asset entirely.

RECOMMENDATION

Consider having the `minimumDeposit` and `depositCap` be configurable by facility as well.

RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Rates Can Change Across Block

CATEGORY Warning
LOCATION YieldDepositFacility.sol
REVIEW Remediation Review

— DESCRIPTION

Function `_takeSnapshot` overwrites vault rates keyed only by `block.timestamp`. Multiple `claimYield()` calls in the same block can record different rates under the same key. Consequently, earlier claims may be underpaid relative to claimants after yield is accumulated in the vault within the block.

— RECOMMENDATION

Clearly document this risk.

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

RESOLVED

Zero Actual Amount Not Verified

CATEGORY Warning
LOCATION BaseDepositFacility.sol: 190
REVIEW Remediation Review

DESCRIPTION

Function `handleCommitWithdraw` checks that the requested `amount_` is non-zero: `if (amount_ == 0) revert DepositFacility_ZeroAmount();`

However, it does not validate that the `actualAmount` is non-zero. If that returned `actualAmount` were to then be used in a token transfer or some other integration, it may lead to unexpected reverts.

RECOMMENDATION

Consider validating that the `actualAmount` is non-zero.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/122>

LOW **L-06**

ACKNOWLEDGED

Min Price Straddle

CATEGORY	Warning
LOCATION	ConvertibleDepositAuctioneer.sol
REVIEW	Remediation Review

— DESCRIPTION

The auction mechanism enforces a daily emission throttle using `_dayState.convertible` and the tick size adjustment logic. As more OHM is sold within a day, tick sizes shrink and prices ramp more aggressively.

However, `_dayState` and `_currentTickSize` are hard-reset during `setAuctionParameters`. This reset clears the global convertible counter and restores tick size to the standard value, regardless of how much OHM was sold just before the reset.

An attacker can exploit this by straddling the reset:

- (1) Place a large bid just before the reset, consuming OHM up to the current capacity at the tick
- (2) Immediately after reset, bid again while the system treats the new day as if no OHM has been sold, with tick size restored.

This allows a single buyer to split a large purchase across two "days," obtaining more OHM at cheaper average prices than if the same purchase had occurred continuously in one day. The intended daily emission throttle is therefore weakened around reset boundaries.

— RECOMMENDATION

Clearly document this behavior.

— RESOLUTION

Status: Acknowledged.

LOW **L-07**

ACKNOWLEDGED

Asymmetry Between Preview And Borrow Amounts

CATEGORY	Rounding
LOCATION	https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/main/src/policies/deposits/DepositRedemptionVault.sol#L435
REVIEW	Remediation Review

DESCRIPTION

The `previewBorrowAgainstRedemption` will calculate principal amount based on redemption details. However, the main `borrowAgainstRedemption` will withdraw assets from vault, incurring in rounding issues when redeeming. This will create asymmetry between the previewed principal amount and the actual tokens received.

RECOMMENDATION

Clearly document this asymmetry,

RESOLUTION

Status: Acknowledged.

LOW **L-08**

ACKNOWLEDGED

Queue Not Cleared When Policy Disabled

CATEGORY	Warning
LOCATION	ConvertibleDepositAuctioneer.sol
REVIEW	Remediation Review

— DESCRIPTION

When the `ConvertibleDepositAuctioneer` policy is disabled, the queue is not cleared. Once the policy is re-enabled, deposit periods which are no longer intended are still processed and affect the deposit count.

— RECOMMENDATION

Consider adding a function to allow governance the ability to adjust the queue as necessary to allow greater fine-tuning of what is in/(ex)cluded. Furthermore, consider if the queue should be cleared when the policy is disabled.

— RESOLUTION

Status: Acknowledged.

LOW **L-09**

RESOLVED

No Single Source Of Truth For Deposit Periods

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/main/src/policies/deposits/ConvertibleDepositAuctioneer.sol
REVIEW	Remediation Review

DESCRIPTION

The `ConvertibleDepositAuctioneer` does not have a single source of truth for deposit periods. In fact, it relies on the 3 different state variables:

- `_depositPeriodsEnabled`
- `_depositPeriods`
- `_depositPeriodsCount`

However, `_depositPeriods` will only contain enabled periods and the count can be easily obtained from the length of the set. Not only this is prone to errors if there is any misalignment, but also consumes more gas.

RECOMMENDATION

Consider having a single source, using the enumerable set.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/124>

INFORMATIONAL **I-01**

RESOLVED

DepositManager_Insolvent Revert Needs More Data

CATEGORY Best Practices
LOCATION DepositManager.sol: 332
REVIEW Remediation Review

DESCRIPTION

Currently the `DepositManager_Insolvent` only contains the `operatorLiabilities` but it would also be useful to view the `depositedSharesInAssets` and `borrowedAmount`.

RECOMMENDATION

Consider adding more data to the `DepositManager_Insolvent` revert

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/123>

INFORMATIONAL I-02

ACKNOWLEDGED

Deposit Periods Can Be Alternated

CATEGORY Warning
LOCATION ConvertibleDepositAuctioneer.sol
REVIEW Remediation Review

DESCRIPTION

In the newly introduced queue, it is possible to alternate deposit period enables/disables continuously to bypass `_validateNoDuplicatePendingChange` checks.

RECOMMENDATION

Be aware and document this behavior.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

ACKNOWLEDGED

Receipt Token Symbol Not Prefixed With "r"

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/main/src/policies/deposits/ReceiptTokenManager.sol#L85
REVIEW	Remediation Review

DESCRIPTION

Previously, the receipt token symbol was prefixed with the letter "r" (i.e. rUSDS3m). Now, the "r" was removed and symbol will just start with the operator name.

RECOMMENDATION

Add the missing prefix "r" to the receipt token symbol.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

RESOLVED

Pending Queue Internal

CATEGORY	Best Practices
LOCATION	ConvertibleDepositAuctioneer.sol: 128
REVIEW	Remediation Review

DESCRIPTION

The `_pendingDepositPeriodChanges` queue is internal but it may be useful for off-chain systems to query which deposit period changes have been queued and will take effect.

RECOMMENDATION

Consider making the queue visibility `public`.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/124>

INFORMATIONAL I-05

RESOLVED

Bond Markets Created When Auctioneer Disabled

CATEGORY	Warning
LOCATION	ConvertibleDepositAuctioneer.sol
REVIEW	Remediation Review

DESCRIPTION

When the Auctioneer policy is disabled, the EmissionManager will continue to `setAuctionParameters` and auction results will be tracked. Consequently, bond markets will continue to be created even though the auctioneer is disabled which may be unexpected.

RECOMMENDATION

Consider if this is expected behavior according to spec.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/125>

INFORMATIONAL I-06

RESOLVED

Superfluous Enumerable Set Validation

CATEGORY Gas Optimization

LOCATION <https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/main/src/policies/deposits/ConvertibleDepositAuctioneer.sol#L635>

REVIEW Remediation Review

DESCRIPTION

The `EnumerableSet.add` will first check if the set contains the value. Therefore, `_enableDepositPeriod` does not require to validate `_depositPeriods.contains(depositPeriod_)` before calling `add`. This will also save some gas and avoid duplicated `contains` call.

RECOMMENDATION

Use the `add` function directly without calling `contains`

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/124>

INFORMATIONAL I-07

RESOLVED

Shadowed State Variable

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/main/src/policies/deposits/ConvertibleDepositAuctioneer.sol#L698>

REVIEW Remediation Review

DESCRIPTION

The `ConvertibleDepositAuctioneer` declares some mapping values and memory variables using the name `isEnabled`. However, this shadows the existing `PolicyEnabler.isEnabled` state variable, potentially causing unexpected issues.

RECOMMENDATION

Consider renaming `isEnabled` used in the `ConvertibleDepositAuctioneer` or prefix a `_` to differentiate it.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/124>

INFORMATIONAL I-08

RESOLVED

Periods Should Be Enabled Before Policy

CATEGORY	Configuration
LOCATION	https://github.com/GuardianOrg/olympus-v3-olympus-team1/blob/main/src/policies/deposits/ConvertibleDepositAuctioneer.sol#L1017
REVIEW	Remediation Review

DESCRIPTION

When auctioneer policy is enabled, it processes pending deposit period changes. If periods are not added to the queue before this policy is enabled, then they will need to wait a full day until `EmissionManager` sets the auction params and process this queue.

RECOMMENDATION

Document this behavior and make sure periods are added to the queue before enabling the policy.

RESOLUTION

Status: Resolved.

<https://github.com/OlympusDAO/olympus-v3/pull/124>

INFORMATIONAL I-09

ACKNOWLEDGED

Backing/premium jumps when Treasury temporarily holds raw reserves

CATEGORY	Warning
LOCATION	https://github.com/0lympusDAO/olympus-v3/blob/dfae803d86eb902c328dd81bbe6162eb69c7cbd8/src/policies/ReserveWrapper.sol#L106-L107
REVIEW	Remediation Review

DESCRIPTION

After this PR, there are windows where the Treasury keeps the raw reserve (e.g., USDS) before periodically depositing into `sReserve` (vault). The bond market's pricing/backing uses `getReserves`, which currently only values `sReserve`. During those windows, idle reserves are excluded, causing stepwise jumps in backing and bond premium, effecting next emissions.

RECOMMENDATION

Beware of this consideration, if not intended, consider include reserve balance inside `getReserves` as well.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-10**

ACKNOWLEDGED

Missing min-shares/slippage check on periodic vault deposits

CATEGORY	Warning
LOCATION	https://github.com/0lympusDAO/olympus-v3/blob/dfae803d86eb902c328dd81bbe6162eb69c7cbd8/src/policies/ReserveWrapper.sol#L106-L107
REVIEW	Remediation Review

DESCRIPTION

Reserve wrapper periodically deposits the entire idle reserve into the vault but doesn't enforce a minimum shares received. This is okay for monotonic, yield-accruing vaults like `sUSDS/USDS`, but would be unsafe for volatile vaults (price can move against the deposit between quote and execution).

RECOMMENDATION

Beware of this consideration, if not intended, consider adding a `minShares` (or `minOut`) parameter to the deposit path and revert on shortfall.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-11**

ACKNOWLEDGED

ERC-6909 supply is not authoritative for receipt tokens

CATEGORY	Warning
LOCATION	https://github.com/0lympusDAO/olympus-v3/blob/dfae803d86eb902c328dd81bbe6162eb69c7cbd8/src/libraries/ERC6909Wrappable.sol#L139-L140
REVIEW	Remediation Review

DESCRIPTION

Users can wrap/unwrap at will, so the `totalSupplies` mapping for the ERC-6909 receipt token won't always reflect true backing or valuation at any instant. Relying on `totalSupply` for economics/accounting can misprice or misreport reserves.

RECOMMENDATION

Do not use ERC-6909 `totalSupply` as a source of truth for valuation/backing.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.